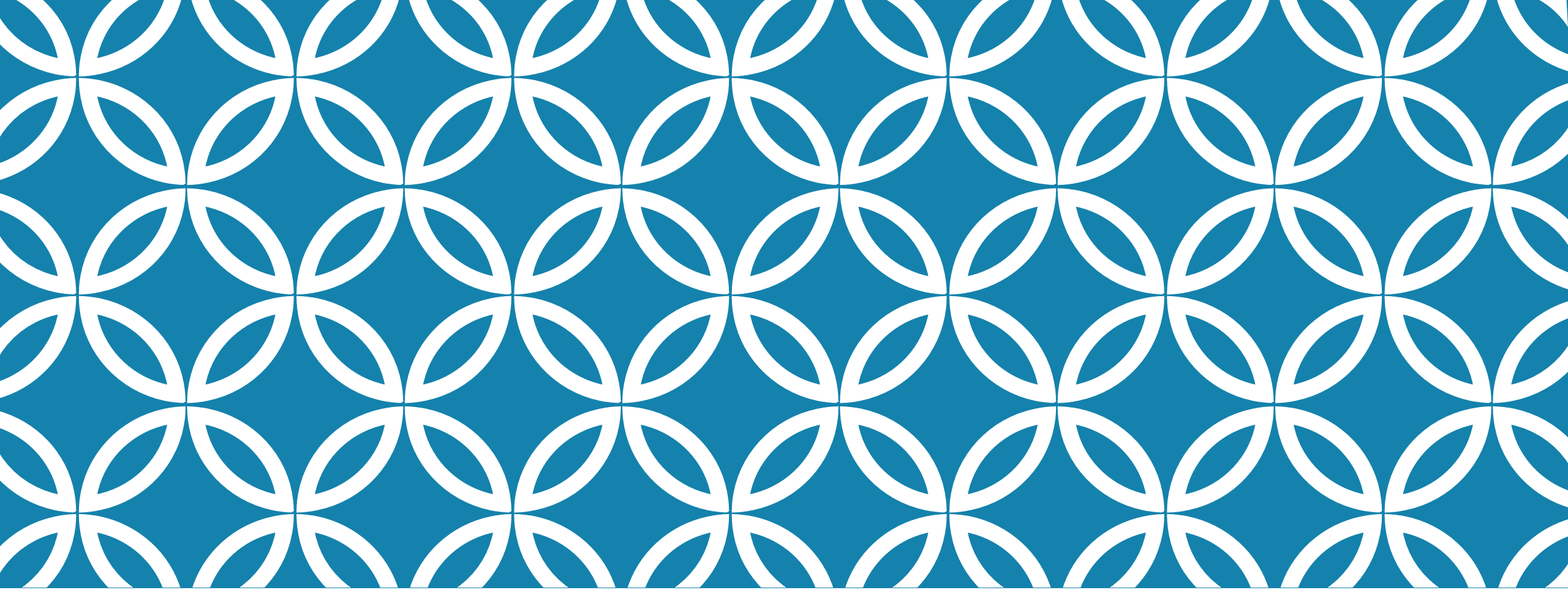


NESNE TABANLI PROGRAMLAMA-I

Öğr. Gör. Erhan AKAGÜNDÜZ

POLYMORPHİSM NEDİR?

- Polymorphism, Türkçe karşılığıyla çok biçimlilik, OOP'nin en önemli kavramlarından biridir.
- Bir metodun veya fonksiyonun farklı nesneler üzerinde farklı davranış göstermesine denir.
- Aynı isimli bir metod, her sınıfta farklı şekilde çalışabilir.
- Örneğin **sound( )** metodu köpekte havlar, kedide miyavlar.
- Bu sayede ortak isimlerle çalışıp farklı sonuçlar alırız.
- Bu yaklaşım yazılımlara esneklik ve genişletilebilirlik kazandırır.

NEDEN POLYMORPHİSM KULLANIRIZ?

- Gerçek hayatta aynı kavram farklı şekillerde gerçekleşebilir.
- Bir insan "ses çıkarır", bir kuş da "ses çıkarır", ama çıkan sesler farklıdır.
- Biz **sesCikar( )** deriz, ama sonuç ***kişiye*** veya ***canlıya*** göre değişir.
- Polymorphism, işte bu esnekliği programlama dünyasına taşır.
- Kodun yeniden kullanılabilirliğini artırır.
- Aynı zamanda miras alma (inheritance) ile birlikte çok güçlü bir yapı sağlar.

PYTHON'DA POLYMORPHİSM MANTIĞI

- Python'da polymorphism, aynı isimdeki metodların farklı sınıflarda farklı işlemler yapmasıyla gerçekleşir.
- Bu sınıfların ortak bir atası olabilir, ama olmak zorunda da değildir.
- Önemli olan, aynı metod isminin farklı davranışlara sahip olmasıdır.
- Bu sayede kodumuz daha okunabilir ve anlaşılır olur.
- Ayrıca kullanıcı, sınıfın detayını bilmeden sadece metod ismini çağırarak işini halleder.
- Şimdi basit bir örnekle inceleyelim.

BASİT POLYMORPHİSM ÖRNEĞİ

- Aynı **sound( )** metodu her sınıfta farklı çalıştı.

```
class Dog:
    def sound(self):
        return "Hav Hav"

class Cat:
    def sound(self):
        return "Miyav"

animals = [Dog(), Cat()]
for animal in animals:
    print(animal.sound())
```

Çıktı:

```
Hav Hav
Miyav
```

POLYMORPHİSM İLE STANDARDİZASYON

- Polymorphism, soyutlamayla (abstraction) beraber kullanıldığında daha güçlü hale gelir.
- Soyut sınıfta belirlenen metod, alt sınıflarda farklı biçimlerde uygulanabilir.
- Böylece tüm sınıflar aynı isme sahip metodu barındırır ama farklı işler yapar.
- Örneğin Animal sınıfında **sound( )** soyut metodunu tanımlarız.
- Alt sınıflar bu metodu doldurmak zorundadır.
- Sonuçta her hayvan için aynı metod çağrılır ama farklı sesler alınır.

KODLA ABSTRACTİON + POLYMORPHİSM

```python
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

class Dog(Animal):
    def sound(self):
        return "Hav Hav"

class Cat(Animal):
    def sound(self):
        return "Miyav"

for a in [Dog(), Cat()]:
    print(a.sound())
```

Çıktı:

```
Hav Hav
Miyav
```

METOD OVERRİDİNG (EZME)

- Polymorphism’in en yaygın kullanımı metod overridingdir.
- Alt sınıflar, üst sınıfta tanımlanan metodları kendine göre yeniden tanımlar.
- Buna **“ezme”** denir, yani aynı isimli metod farklı şekilde çalışır.
- Örneğin üst sınıfta **bilgi( )** metodu genel bilgi verirken, alt sınıflar kendi bilgilerini verebilir.
- Bu, kodun daha esnek ve anlamlı olmasını sağlar.
- Şimdi örnek kod görelim.

METOD OVERRİDİNG ÖRNEĞİ

```
class Person:
    def info(self):
        return "Ben bir insanım."

class Student(Person):
    def info(self):
        return "Ben bir öğrenciyim."

class Teacher(Person):
    def info(self):
        return "Ben bir öğretmenim."

for p in [Person(), Student(), Teacher()]:
    print(p.info())
```

Çıktı:

```
Ben bir insanım.
Ben bir öğrenciyim.
Ben bir öğretmenim.
```

FONKSİYONLARDA POLYMORPHİSM

- Polymorphism sadece sınıflarda değil, fonksiyonlarda da uygulanabilir.
- Bir fonksiyon farklı veri tipleriyle çalışacak şekilde yazılabilir.
- Python dinamik bir dil olduğu için parametrelerin tipini önceden belirtmeyiz.
- Aynı fonksiyon sayılarla toplama yapabilirken, **stringlerle** birleştirme de yapabilir.
- Bu durum da aslında bir polymorphism örneğidir.
- Şimdi hemen görelim.

FONKSİYON ÖRNEĞİ

- Burada aynı + operatörü hem toplama hem de string birleştirme yaptı.
- İşte bu da operatör düzeyinde bir polymorphism örneğidir.

```
print(5 + 10)       # sayıların toplamı
print("Ali" + "Veli")  # string birleştirme
```

Çıktı:

```
15
AliVeli
```

OPERATÖR OVERLOADİNG

- Python’da operatörlerin davranışını sınıflar üzerinde değiştirebiliriz.
- Buna “**operator overloading**” denir ve polymorphism’in özel bir kullanım alanıdır.
- Mesela + operatörünü kendi sınıfımıza göre yeniden tanımlayabiliriz.
- Böylece iki nesne toplandığında istediğimiz sonucu alırız.
- Bu yöntem, kodları daha doğal ve okunabilir hale getirir.
- Şimdi bir örnek görelim.

OPERATÖR OVERLOADİNG ÖRNEĞİ

- Aynı + operatörü farklı davranış sergiledi.

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other.value

n1 = Number(10)
n2 = Number(20)
print(n1 + n2)
```

Çıktı:

```
30
```

POLYMORPHİSM AVANTAJLARI

- Kod tekrarını azaltır.
- Ortak metod isimleriyle kullanım kolaylığı sağlar.
- Ekip çalışmasında standardizasyon oluşturur.
- Soyutlama ve miras ile birleştiğinde güçlü bir yapı kurar.
- Esnekliği artırır, yeni sınıflar kolayca eklenebilir.
- Gerçek hayattaki **“çok biçimlilik”** yapısını yazılıma taşır.

POLYMORPHİSM DEZAVANTAJLARI

- Yanlış tasarlanırsa kafa karışıklığına yol açabilir.
- Küçük projelerde gereksiz karmaşıklık oluşturabilir.
- Hataların izini sürmek daha zor olabilir.
- Fazla soyutlama ile birlikte kullanıldığında öğrenme eğrisi artabilir.
- Yeni başlayanlar için kavram kargaşası yaşatabilir.
- Ancak doğru kullanıldığında bu dezavantajlar ortadan kalkar.

DERS ÖZETİ

- Polymorphism, OOP'nin en esnek ve güçlü özelliklerinden biridir.
- Aynı isimdeki metodların farklı sınıflarda farklı çalışmasını sağlar.
- Metod overriding, operator overloading ve fonksiyon çok biçimliliği en yaygın örnekleridir.
- Soyutlama ile birlikte kullanıldığında, projelere standart ve düzen kazandırır.
- Kullanıcıya kolaylık sağlarken geliştiriciye esneklik sunar.